PHASE

15

Hardening, Polish and Release: Turk-OS 1.0

Make it survive hostile input, measure it, document it, ship it, and choose where to go next

Overall progress toward Turk-OS 1.0 (this phase in red) 96 % → 100 %

Duration	3–4 weeks for the core, then open-ended
Difficulty	★★★★☆ (4/5)
You need	Phases 0–14 complete: Turk-OS boots to a shell with pipes, files and processes
You will have	A system call fuzzer that runs for an hour without a kernel panic, an audited user/kernel boundary, kernel stack protection and read-only kernel code, users, file and device permissions with sessions, a threat model with its residual risks, a benchmark suite with one measured optimisation, an architecture document, a reproducible `make release` that produces a bootable ISO with published hashes, the signed `v1.0` tag, and a written reflection on your design. After that, a choice of tracks beyond 100 %, and Stage V: the RISC-V and ARM ports of Phases 16–18.

1 Why this phase

A system isn't finished when its features work on the inputs you tried. It is finished when it survives inputs nobody intended, when you know how fast it is and why, when someone else can build and understand it, and when you can explain every design choice in it. That is the difference between a demo and version 1.0, and this phase is about closing that gap.

It is also a good moment to step back. MOS chapter 12, written by an author who designed several operating systems, is about exactly the decisions you have been making for nine months: interfaces, structure, mechanism versus policy, performance, and managing a large software project. Reading it now, with your own kernel in mind, is very different from reading it at the start. The phase ends with a written comparison of Turk-OS against MINIX 3 and Linux, a list of optional tracks for going further, and a look ahead to Stage V, which ports Turk-OS to RISC-V (including the core you are building in hardware) and to ARM.

2 Concepts you need

2.1 The kernel is the trusted computing base

MOS §9.2.2 defines the **trusted computing base** (TCB): the hardware and software that must be correct for the system's security to hold. In Turk-OS that is the entire kernel. Every system call is an entry point into the TCB controlled by a possibly hostile program, which is why Phase 10's rule (never trust a user pointer) was only the start. MOS §9.7 catalogues how kernels get broken, and several items apply directly to code you have written: **buffer overflows** when a length from the user isn't checked; **integer overflows** when `offset + length` wraps around; **null-pointer dereferences** in the kernel; and **time-of-check to time-of-use** (TOCTOU) races, where the kernel checks a value in user memory and then reads it again after the user has changed it. The cure for the last one is to copy first and check the copy.

2.2 Protection: who may do what

Protection needs a model of *subjects* (users, processes), *objects* (files, devices) and *rights* (read, write, execute). MOS §9.3 and OSC chapter 17 describe the two ways to store it: **access control lists** attached to objects, and **capabilities** held by subjects. UNIX uses a compact ACL, nine permission bits for owner, group and others in each inode, plus the rule that user ID 0 (root) may do anything. TurkFS already stores the mode, user ID and group ID in every inode (Phase 13), so what is left is to give processes an identity and check it.

2.3 Measuring before optimising

MOS §12.4 gives the rules for performance work: find out what is actually slow, optimise the common case, use caching and hints, and exploit locality. None of that works without measurements. The x86 `rdtsc` instruction reads a cycle counter, which is precise enough to time a single system call. The classic operating-system microbenchmarks (in the style of *lmbench*) measure the null system call, a context switch between two processes passing a token through pipes, process creation, and file-system throughput.

2.4 Design, looking back

MOS §12.2–12.3 lists principles you can now test against your own code: **simplicity** and **completeness** of the interface, **orthogonality** (independent concepts that combine freely, like `fork` and `exec`), **separation of mechanism and policy** (your scheduler), static versus dynamic structures (MINIX's fixed tables versus your heap), and top-down versus bottom-up implementation (you went bottom-up). §12.5 adds Brooks's observations about software projects, including the *second-system effect*: the temptation to put everything you left out of version 1 into version 2.

3 Reading plan

Priority	Book	What to read	Pages	Why now
REQUIRED	MOS	Ch. 12 *Operating System Design*: nature of the problem, interface design, implementation, performance, project management, trends	981–1030	Read all of it, with Turk-OS in mind.
REQUIRED	MOS	§9.1–9.3 *The Security Environment*, *Operating Systems Security*, *Controlling Access to Resources*	595–611	Threats, the TCB, protection domains, ACLs and capabilities.
REQUIRED	MOS	§9.7 *Exploiting Software* (buffer overflows, format strings, dangling and null pointers, integer overflows, TOCTOU)	639–657	The bugs your audit is looking for.
RECOMMENDED	MOS	§9.10 *Defenses* (especially code signing and jailing)	684–703	What operating systems add on top of correct code.
RECOMMENDED	OSC	§16.1–16.2 *The Security Problem*, *Program Threats*; §16.5–16.6 *User Authentication*, *Implementing Security Defenses*	621–634, 648–662	A second view, with passwords and defences in depth.
RECOMMENDED	OSC	Ch. 17 *Protection*, §17.1–17.6 (principles, rings, domains, access matrix)	667–682	The formal model behind UNIX permissions.

Priority	Book	What to read	Pages	Why now
RECOMMENDED	OSDI	§5.4–5.5 *Security*, *Protection Mechanisms*	526–548	Compact coverage of authentication, ACLs, capabilities and covert channels.
RECOMMENDED	OSDI	§1.5 *Operating System Structure*; §2.5.1 *The Internal Structure of MINIX 3*	42–51, 112–116	Monolithic versus microkernel, for your design reflection.
RECOMMENDED	MOS	§10.1–10.8 *Case Study: UNIX, Linux, and Android* (skim)	713–848	Compare each subsystem with yours.
OPTIONAL	OSC	Ch. 20 *The Linux System*	775–819	Another view of Linux for the comparison.
RECOMMENDED	LKD	Ch. 18, *Debugging*	363–378	Oopses, assertions and debugging techniques for your hardening work.
OPTIONAL	LKP	Ch. 5, *Kernel modules and security – an overview*; Ch. 7, *Randomizing the memory layout – KASLR*	256–261, 362–367	Defences a production kernel adds.
RECOMMENDED	SPEC	NIST SP 800-218, *Secure Software Development Framework* (`https://csrc.nist.gov/pubs/sp/800/218/final`), practices PS and PW	–	The yardstick for Task 15.7's release integrity and Task 15.2's review.
RECOMMENDED	SPEC	Linux kernel documentation, *Kernel Self-Protection* (`https://docs.kernel.org/security/self-protection.html`)	–	The defences of Task 15.3, and the ones Turk-OS leaves out.

Real-system lens: how MINIX 3 does it

MINIX 3's central design goal is reliability, and OSDI §2.5.1 explains how its structure serves it. Drivers run as separate user processes with only the privileges they need, so a buggy driver can't overwrite the kernel. A special process, the **reincarnation server**, watches the drivers and servers and restarts any that crash (OSDI pp. 114 and 599). In Turk-OS, every driver bug is a kernel bug. Keep this contrast in mind for Task 15.8, and look at Track E below if you want to try the MINIX approach on one of your own drivers.

Linux, for comparison. LKD LKP Linux's hardening goes further: KASLR, signed modules, lockdep and memory-error detectors such as KASAN, plus the `BUG_ON`/`WARN_ON` assertions and oops reports that LKD's chapter on debugging describes.

4 Build it

Task 15.1 Fuzz the system call interface

`user/bin/fuzz` has made random system calls since Phase 11, with a seed you can print and replay, and Phase 14 taught it pipes and the terminal. This is its final pass. Check that the argument mix covers every class: small integers, huge integers, negative numbers, pointers to valid buffers, pointers into the kernel (`0xC0000000` and up), pointers into the program's own read-only text, unmapped addresses, addresses one byte before the end of a page, descriptors that exist and ones that don't, and random strings as paths. Add each call of this phase as you write it. Run several copies at once together with a stress script that forks, execs, pipes and writes files in parallel, first as root and, once Task 15.4 exists, as an ordinary user (UID 1000). Every kernel panic is a bug to fix; every process

killed by a signal is fine. If you have a spare machine, also let the campaign soak for 24 hours locally; hosted CI jobs are time-limited, so the long run stays out of CI.

Verify: Four fuzzers and the stress script run for one hour with zero kernel panics and no growth in memory use after they are stopped, both as root and as UID 1000.

Task 15.2 Audit the boundary

The boundary table `docs/security/syscalls.md` has grown since Phase 10, one row per system call with each argument and how it is validated. Make the final pass: every call, including those of this phase, has a complete row, the code still does what the row says, every user pointer goes through the uaccess routines, and `cppcheck` runs clean. Then look specifically for these bugs, because the fuzzer is unlikely to find all of them.

Bug class	Where to look in Turk-OS
missing pointer check	any use of a user pointer outside `copy_from_user/copy_to_user/strncpy_from_user`
integer overflow	`offset + len` in `read/write/lseek`, `brk` arguments, ELF segment sizes, `argv` totals
TOCTOU	paths or structures read twice from user memory; copy once into the kernel, then validate the copy
unchecked index	descriptor numbers, signal numbers, system call numbers, directory entry indices
information leak	kernel stack or heap bytes copied to the user in padding of `struct stat` or `dirent`; zero structures before filling them
resource exhaustion	unlimited processes, descriptors, pipe buffers or path lengths; every resource needs a limit

Task 15.3 Harden the kernel itself

The kernel has been built with `-fstack-protector-strong` since Phase 10, and its `.text` and `.rodata` have been read-only since Phase 6. Check both once more on the release build: every kernel object file gets the protector except the one function that sets `__stack_chk_guard`, which is compiled with `-fno-stack-protector` and never returns into a frame entered before the guard changed (that return would report “stack smashing” on a correct stack). The kernel image is still mapped with 4 KiB pages, CR0.WP is still set, and the page at address 0 is never mapped in any process.

The guard is only as good as its seed, and `rdtsc` alone is predictable. Give the kernel one entropy function: `rdrand` when CPUID reports it (QEMU’s `-cpu max` does), otherwise `rdtsc` mixed with the Multiboot memory map. Write in the threat model that the fallback is weak and acceptable only for a teaching system.

kernel/arch/i386/random.c

```
#include <cpuid.h>

uint32_t arch_random32(void)
{
    unsigned a, b, c, d;
    if (__get_cpuid(1, &a, &b, &c, &d) && (c & bit_RDRND)) {
        for (int i = 0; i < 10; i++) {          /* rdrand may fail briefly; retry */
            uint32_t v;
            unsigned char ok;
            __asm__ volatile("rdrand %0; setc %1" : "=r"(v), "=qm"(ok) :: "cc");
            if (ok)
                return v;
        }
    }
    return weak_seed();                         /* rdtsc ^ memory map hash: predictable */
}
```

Two CPU features cost one bit each. When CPUID leaf 7 reports **SMEP** (EBX bit 7), set CR4 bit 20: the CPU then refuses to execute a user page in ring 0, so a corrupted kernel function pointer can't jump into code a program wrote. **SMAP** (EBX bit 20, CR4 bit 21) is optional. It also blocks ring-0 data access to user pages, so the uaccess routines must wrap their copy in `stac` and `clac`; with a single uaccess choke point that is a two-line change.

Security denials should leave a trace. Log a failed `setuid`, a permission denied on a device node and a process killed by a fault with `pr_warn` and the prefix `security:`, at that one fixed level, so one search of `log_buf` for `security:` answers "what was refused?". Finally, make crash reports readable: generate a symbol table from `i686-elf-nm` at build time, load it as a boot module, and print function names in the backtrace of every panic.

Verify: A deliberate overflow of a local buffer in a test system call panics with "stack smashing detected" and a symbolised backtrace; a write to a function's code faults. Under `-cpu max` the boot log reports `rdrand` and SMEP, and a test system call that jumps to a user address faults instead of running it.

Task 15.4 Users and permissions

Give each task a user ID and a group ID, inherited through `fork`. Add `getuid` (24), `setuid` (23), `getgid` and `setgid`; only root may change identity. Fill in `vfs_permission` (Task 13.1), which every VFS operation already calls, and check permissions there against the inode's mode, owner and group on `open`, `execve`, `create`, `mkdir` and `unlink`, with root bypassing the checks. The same checks cover `rename`, `link`, `chmod`, `chown` and `truncate`, including `open` with `O_TRUNC`; only the owner or root may `chmod`, and only root may `chown`.

Files are not the only objects. A user who can open `/dev/hda` can rewrite every inode, `/etc/passwd` included, and walk past all the checks above. Give each `devfs` node an owner, a group and a mode, and check them through the same VFS path: `/dev/hd*` is `root:root 0600`, `/dev/tty` is `0620`, and `/dev/null` and `/dev/zero` are `0666`.

The terminal needs an owner too. As Phase 14 left it, `ioctl` 54 lets any process make any group the foreground, so a background job left behind by one user could take the console and read the next user's `login` input. Give each process a session ID, inherited through `fork`, and add `setsid`, which makes the caller the leader of a new session. The first terminal a session leader opens becomes the session's **controlling terminal**. `ioctl` 54 then accepts only a process group in the caller's own session and returns `-EPERM` otherwise. When the session leader exits, the kernel sends `SIGHUP` to every process in the session and revokes their descriptors on the terminal, so later reads and writes on them fail.

Add `/etc/passwd` with lines of the form `name:uid:gid:home:shell`, and a `login` program that `init` runs on the console, which looks up the user, starts a new session, changes to their identity and home directory, and runs their shell. `login` opens `/etc/passwd` and every other file of its own with `O_CLOEXEC`, so no descriptor opened as root survives into the user's shell. It drops privileges in the safe order, group first, because after `setuid` it may no longer change its group, and it stops if either call fails.

user/bin/login.c (dropping privileges)

```
if (setgid(pw->gid) < 0 || setuid(pw->uid) < 0) {
    fprintf(stderr, "login: cannot change identity\n");
    exit(1);                    /* never run the shell as root by accident */
}
```

Then decide about passwords and write the decision down. One honest option is no passwords at all: `login` only asks for a name, and the threat model states that whoever has the console is trusted, which is true of a QEMU window. The other is real passwords, and then never write your own hash. Port the reference code of a deliberately slow, salted key-derivation function (bcrypt, scrypt or

Argon2) into user space, compute the hashes on the host when you build the image, and store them in `/etc/shadow`, owned by root with mode `0600`, because `/etc/passwd` must stay readable by every user. A plain salted SHA-2 hash is too fast and is brute-forced in hours.

Verify: An ordinary user can't read a file with mode `0600` owned by root, can't write to `/bin`, and can't kill root's processes; root can do all three. As UID 1000, every evil program still behaves as listed, `open` of `/dev/hda` returns `-EACCES`, and a background job's `ioctl` 54 on another session's terminal returns `-EPERM`. After logout, a job left running in the old session has received `SIGHUP`.

Task 15.5 Benchmarks, and one real optimisation

Write `user/bin/bench`, which reports the median of many runs for each measurement below, and put the results in a table in your documentation. Then pick the worst number, find the cause with measurements, fix it, and record the before and after values.

Benchmark	How
null system call	time 100 000 calls to `getpid` with `rdtsc`
context switch	two processes pass a byte back and forth through two pipes; divide by the number of switches
process creation	`fork` + `exit` + `wait`; and `fork` + `exec` + `wait` of a tiny program
file throughput	write and read a 16 MiB file on TurkFS, with a cold and a warm buffer cache
boot time	`rdtsc` in the loader and at the first shell prompt, printed on the serial port

Typical optimisations at this point are global pages for kernel mappings (no TLB flush of kernel entries on a CR3 switch), multi-sector disk transfers and read-ahead, a hash table in the buffer cache, and avoiding the double copy through a kernel buffer in pipes. Run the same benchmarks in a Linux system for perspective; the point is to understand the differences, not to win.

Task 15.6 Write the documentation

Write an architecture document (a LaTeX file next to this plan, or Markdown): an overview diagram like the one in the roadmap; each subsystem in a page or two; the virtual and physical memory maps; the system call table with numbers, arguments and errors; the lock order; the TurkFS on-disk format; how to build, run, test and debug; and known limitations. Add a `README` that gets a newcomer from a clean clone to a running shell, and a license (MIT matches your other open-source work).

Bring `docs/security/threat-model.md` to version 1. Next to the assets, adversaries and trust boundaries it already lists, add a table of residual risks: each threat, the control that addresses it, and what is left, such as the weak entropy fallback, the missing W^X for user data on i386 without PAE, or console access trusted when there are no passwords. Close it with an explicit out-of-scope list: physical attackers and the boot chain (BIOS boot has no root of trust), network attackers (there is no network stack), malicious hardware and DMA, and speculative-execution side channels such as Spectre and Meltdown. Writing them down is what separates a decision from an oversight. Add `SECURITY.md` at the top of the repository: which version is supported, how to report a bug privately, and a link to the threat model.

Task 15.7 Release v1.0

Add `make release`: build the kernel, the user programs, the root archive and a TurkFS image populated with `/bin`, `/etc` and a home directory, and a GRUB ISO whose `grub.cfg` loads the kernel and its modules. Check that the release builds from a clean clone (`git clone` into a temporary directory, then `make release`), boot the ISO in QEMU, and tag the commit `v1.0`. If you want to try real hardware, an older PC with legacy BIOS (or UEFI with CSM) can boot the ISO from a USB stick.

Writing a USB stick

`sudo dd if=build/turkos.iso of=/dev/sdX bs=4M` erases the target device completely. Run `lsblk` before and after plugging the stick in, make sure `sdX` is the stick and not your laptop's disk, and never guess.

"Reproducible" is a claim you can test. Fix the time the build embeds with `SOURCE_DATE_EPOCH`, keep the clone's path out of debug information with `-ffile-prefix-map`, build twice from two clean clones, and compare the hashes. `kernel.elf` and the user binaries must be identical. `grub-mkrescue` may embed timestamps in the ISO; if two ISOs differ, say so in the release notes instead of hiding it. Publish a `SHA256SUMS` file with the ISO, the kernel and the TurkFS image, and sign the tag so anyone can check that it came from you.

Two clean builds, hashes and a signed tag

```
export SOURCE_DATE_EPOCH=$(git log -1 --format=%ct)
git clone -q . ../rel-a && make -C ../rel-a release
git clone -q . ../rel-b && make -C ../rel-b release
sha256sum ../rel-a/build/kernel.elf ../rel-b/build/kernel.elf    # the two hashes must match
(cd ../rel-a/build && sha256sum turkos.iso kernel.elf turkfs.img > SHA256SUMS)
git tag -s v1.0 -m "Turk-OS 1.0"                                  # needs a GPG key
```

Verify: A fresh clone builds with one command and boots from ISO to the login prompt. `make test` passes on the tagged commit. Two clean builds give identical `kernel.elf` hashes, `SHA256SUMS` matches the published files, and `git tag -v v1.0` reports a good signature.

Task 15.8 Defend your design

Write a reflection of three to five pages, as if for a thesis committee. For each subsystem (boot, memory, processes, scheduling, synchronisation, system calls, file system, drivers), state what Turk-OS does, how MINIX 3 does it (OSDI) and how Linux does it (MOS chapter 10), and what you would change with hindsight. Use MOS chapter 12's principles as the yardstick, and end with the one decision you are proudest of and the one you would reverse first. Presenting it to your advisor or classmates is the best test of whether you really understand your own system.

5 Beyond 100 %: choose a track

Turk-OS 1.0 is a complete small system. The guided way on is Stage V (Phases 16–18), which ports it to two more architectures with tasks, checks and milestones like the ones you know. The tracks below are open-ended instead. They extend Turk-OS in different directions; each is a project of several weeks to several months, and they can be combined with each other and with Stage V. Pick by interest, not by size.

Track	What you build	Where to read
A. Graphics and Turkish text	Request a linear framebuffer through the Multiboot video fields, draw text with a PSF font that covers the whole Turkish alphabet (ğ, ş, ı, İ, Ğ, Ş), add a PS/2 mouse (IRQ 12), then a simple window system	MOS §5.6 (user interfaces); OSDev Wiki (VESA, PSF, mouse)
B. Multiprocessor (SMP)	Parse the ACPI MADT, program the local APIC and I/O APIC, start the other CPUs (INIT–SIPI–SIPI), per-CPU data and run queues, TLB shootdown; your Phase 9 spinlocks finally spin	MOS §8.1 (520–544); OSTEP Ch. 10 (97–108); OSC §5.5 (220–227)

Track	What you build	Where to read
C. Networking	PCI enumeration, an e1000 or RTL8139 driver in QEMU, Ethernet, ARP, IPv4, ICMP (`ping`), UDP, then a minimal TCP and a socket API	OSC Ch. 19 (733–768); MOS §8.3 (566–587); OSDev Wiki and RFCs
D. 64-bit port	Long mode, four-level paging, Multiboot2, `syscall`/`sysret`, the x86-64 calling convention. Much easier after Stage V: Phase 16 makes the generic code 64-bit clean, and Phase 18 builds a 64-bit MMU on AArch64	OSC §9.6 (379–383); Intel SDM Vol. 3A
E. Microkernel experiment	Move the ATA driver (or TurkFS) into a user-space server that talks to the kernel with synchronous messages, as MINIX does, and measure the cost	OSDI §2.5, §3.4 (112–125, 252–261); MOS §1.7.3 (65–68)
F. Virtualisation	Understand what QEMU and KVM do for you, run Turk-OS with `-enable-kvm`, and compare the benchmarks	OSC Ch. 18 (701–729); MOS Ch. 7 (471–516)
G. Loadable kernel modules	Load relocatable ELF objects (`.ko`-style) into a running kernel: resolve their symbols against an exported symbol table, apply relocations, call `init` and `exit`, and move one driver (the RAM disk) into a module, with `insmod`, `rmmod` and `lsmod` commands	LKP Ch. 4–5 (143–265); LKD Ch. 17 (337–362)

Stage V deserves a note. Porting forces you to find every place where x86 leaked into supposedly generic code, and fixing those leaks improves the whole kernel, including the x86 version. Running your own operating system on a processor you designed yourself is a rare thing to be able to say about a student project, and a natural thesis topic. Stage V has its own progress scale, from 0 to 100 % toward Turk-OS 2.0, so the 100 % you reach in this phase stays what it is: Turk-OS 1.0, finished.

6 Milestone

Milestone: Turk-OS 1.0

- ☐ One hour of parallel fuzzing and stress testing, as root and as UID 1000, without a kernel panic or a memory leak.
- ☐ Boundary audit table complete; stack protector, read-only kernel code and symbolised backtraces in place.
- ☐ Entropy source, SMEP when available, and `security:` log lines for denials in place.
- ☐ Users, `login` and file permissions enforced; device nodes owned, sessions and terminal ownership enforced; the password decision written down.
- ☐ Benchmark table written, with one optimisation measured before and after.
- ☐ Architecture document, README and system call reference written; threat model v1 lists every residual risk; `SECURITY.md` in place.
- ☐ `make release` works from a clean clone; the ISO boots; two clean builds give identical `kernel.elf` hashes; `SHA256SUMS` is published and `v1.0` is a signed tag.
- ☐ The design reflection is written, and ideally presented.

Self-check questions

1. Which parts of Turk-OS trust input from user programs, and how is each validated?
2. Give an example of a TOCTOU bug that could exist in `execve`, and explain how copying first prevents it.
3. Where would MINIX 3 put your ATA driver, and what would change about a driver bug's consequences?
4. What is the most expensive operation you measured, and where does the time go?
5. Which of MOS chapter 12's principles did Turk-OS follow best, and which did it violate most?
6. What would break first if Turk-OS ran on two CPUs tomorrow?

7 Closing

You started this plan knowing basic C and no assembly. Turk-OS 1.0 boots on a PC, protects processes from each other, schedules them preemptively, manages virtual memory with copy-on-write, stores files in a real on-disk format, and runs a shell with pipes. More useful than the code is what you now understand: what happens between pressing a key and a character appearing, between `fork` and `exit`, between `write` and the disk. The five books on your shelf will read differently from now on, because every chapter describes something you have built or decided not to build. Keep the journal going, pick a track, or turn the page to Stage V.

Next: **Phase 16 Portability: an Architecture Layer** →

Books used in this plan

LOB Erik Helin, Adam Renberg. *The little book about OS development*, 2015. The hands-on x86 guide. Free at https://littleosbook.github.io.

OSTEP Remzi H. Arpaci-Dusseau, Andrea C. Arpaci-Dusseau. *Operating Systems: Three Easy Pieces*, v0.90, Arpaci-Dusseau Books, 2015. Main theory text. Free at https://pages.cs.wisc.edu/~remzi/OSTEP/.

OSC A. Silberschatz, P. B. Galvin, G. Gagne. *Operating System Concepts*, 10th ed., Wiley, 2018. Reference theory text.

MOS Andrew S. Tanenbaum, Herbert Bos. *Modern Operating Systems*, 4th ed., Pearson, 2015. Hardware, I/O, design, security.

OSDI Andrew S. Tanenbaum, Albert S. Woodhull. *Operating Systems: Design and Implementation*, 3rd ed., Pearson Prentice Hall, 2006. MINIX 3 with full source listing.

LKD Robert Love. *Linux Kernel Development*, 3rd ed., Addison-Wesley, 2010. How Linux implements each subsystem.

LKP Kaiwan N. Billimoria. *Linux Kernel Programming*, Packt, 2021. Modern kernel practice: logging, allocators, the scheduler, synchronisation.

The books do not come with this plan. LOB and OSTEP are free online; buy or borrow the other five (the roadmap lists their ISBNs). Where books disagree, the plan follows the newer one when its method is also better for Turk-OS. Page numbers are the *printed* page numbers of these editions, not PDF page indices. With another edition, or with the free OSTEP chapters (each one starts at page 1), find the reading by its chapter and section numbers. MINIX source references like `kernel/proc.c (07400)` give the file and its starting line in OSDI Appendix B.